Batch: B2 Roll No.:16010124107
Experiment No. 03
Grade: AA / AB / BB / BC / CC / CD /DD
Signature of the Staff In-charge with date

TITLE: System calls

AIM: To understand the working Process based system calls.

Expected Outcome of Experiment:

CO 1. To introduce basic concepts and functions of operating systems.

Books/ Journals/ Websites referred:

1. **Silberschatz A., Galvin P., Gagne G. "Operating Systems Principles", Willey Eight edition.**
2. **William Stallings "Operating Systems" Person, Seventh Edition Edition.**
3. **Sumitabha Das " UNIX Concepts & Applications", McGraw Hill Second Edition.**

Pre Lab/ Prior Concepts:

System Calls Provide the Interface between a process and the OS.

System calls are usually made when a process in user mode requires access to a resource.

Then it requests the kernel to provide the resource via a system call.

System calls are required in the following situations –

1) If a file system requires the creation or deletion of files.

2) Reading and writing from files also require a system call.

3) Creation and management of new processes.

4) Network connections also require system calls. This includes sending and receiving packets.

5) Access to a hardware devices such as a printer, scanner etc. requires a system call.

Description of the application to be implemented:

Program for System Call:

1. Write a Program for creating a process using System call (E.g fork()). Create a child process. Display the details about that process using getpid and getppid functions in each block. Also print the return value of fork() system call in each block. In the child process, Open a text file using file system calls and read the contents of the text file and display it.

```cpp
#include <fstream>
#include <iostream>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

using namespace std;

int main() {

    int f = fork();

    if (f < 0) {
        cout << "Fork failed\n";
        return 1;
    }
    else if (f == 0) {
        cout << "Child Process\n";
        cout << "Child PID: " << getpid() << endl;
        cout << "Parent PID: " << getppid() << endl;
        cout << "Enter the name of the file you want to open\n";
        string filename;
        cin >> filename;
        ifstream file(filename);
        if (!file) {
            cout << "Error opening file\n";
        } else {
        string line;

            cout << "File 'text.txt' opened successfully: " << endl;
            while (getline(file, line)) {
            cout << line << endl;
        }
        } else {
        string line;

            cout << "File 'text.txt' opened successfully: " << endl;
            while (getline(file, line)) {
            cout << line << endl;
        }

file.close();
    }
    }
    else {
        // Parent process
        wait(NULL);
        cout << "Parent Process\n";
        cout << "Child PID: " << f << endl;
        cout << "Parent PID: " << getpid() << endl;
    }
}
```

Implementation details: (Screen shots)

```
kjsce@kjsce-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~$ ./output
Child Process
Child PID: 13528
Parent PID: 13527
Enter the name of the file you want to open
text.txt
File 'text.txt' opened successfully:
Hello world!
Parent Process
Child PID: 13528
Parent PID: 13527
```

Conclusion :

This experiment was able to demonstrate the purpose and working of the fork() function and how the PID of parent and child differs from a child process and a parent process.

Post Lab Descriptive Questions

1) Describe System Call Interface.

The System Call Interface (SCI) is the crucial boundary and communication mechanism that allows a user-level computer program to request services from the operating system's kernel.

2) List the types of System Calls.

There are five types of system calls:
1. Process Control
2. File Management
3. Device Management
4. Information Maintenance,

5. Communication

Date: 11/02/2026 **Signature of faculty in-charge**